this ideal are using components that can never fail or accounting for every possible fault by providing a recovery action, which is also assured of success. In most architectures, what you have instead is a catch-all mechanism that terminates as soon as an uncaught failure arises.

Even if an application attempts to provide recovery strategies, testing them is hard, and ensuring that the recovery strategies work adds another layer of complexity. In the procedural world, each attempt to do something requires a return code that is checked against a list of possible faults. Exception handling has become a fixture of modern languages, promising a less onerous path to providing the various required means of recovery. Although it has succeeded in yielding code that doesn’t need fault checks on every line, the propagation of faults to ready handlers hasn’t improved significantly.

The idea of a system that’s free of faults sounds great in theory, but the sad fact is that building a system that’s also highly available and distributed is not possible for a nontrivial system. The main reason is that large parts of any nontrivial system aren’t under your control, and these parts can break. Then there’s the prevalent problem of responsibility: as collaborators interact, often using shared components, it’s not clear who’s responsible for which possible faults.

A good example of potentially unavailable resources is the network: it can go awry at any time or be partly available. If you want to continue operation, you have to find another way to communicate or disable communication for a while. You may depend on third-party services that can misbehave, fail, or be sporadically unavailable. The servers your software runs on can fail, be unavailable, or experience total hardware failure. You obviously can’t magically make a server reappear out of its ashes or automatically fix a broken disk to